

Edge Authentication & Centralized Authorization

Kong (OIDC plugin) for authN + a single auth-service for authZ — investigation spike & RFC

SPIKE / RFC

OPEN — DISCOVERY

PRIORITY: P2

Issue type	Spike (time-boxed investigation) — proposes an RFC for review
Key	PLAT-XXX
Epic / Initiative	Platform AuthN/AuthZ consolidation
Components	api-gateway, identity, platform-infra
Labels	spike oidc kong keycloak rfc
Reporter	freeman@promotingqueens.org
Assignee	TBD
Story points / timebox	5 pts · 3 days timebox
Reference PoC	sso-demo-3/ (runnable; this ticket is derived from it)

Summary

Investigate moving authentication to the **edge gateway** and centralizing authorization in a **single policy service**, so application services hold *no* auth code. Concretely: **Kong's OIDC plugin** authenticates every request against **Keycloak** (which brokers to Microsoft Entra ID); Kong then forwards to an **auth-service** that evaluates a central policy and proxies allowed requests to the apps. Produce a recommendation (adopt / adopt-with-changes / reject) plus the open questions, risks, and a reference implementation.

Background & current state

We have validated three patterns in the `sso-demo*` PoCs. This spike evaluates the third for production adoption.

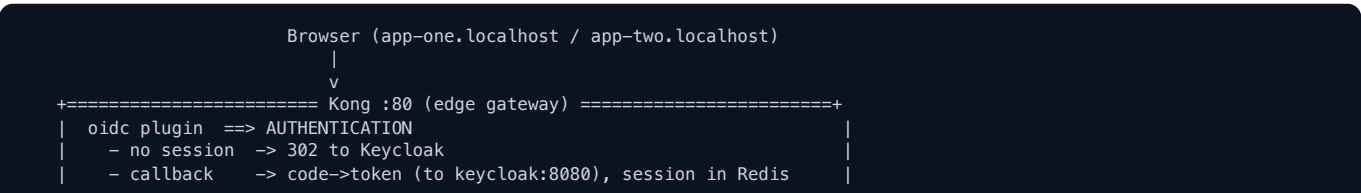
Pattern	OIDC client	AuthN	AuthZ	App auth code
In-app OIDC	each backend	each app	each app	lots, duplicated
Gateway authN	Kong plugin	gateway	each app (role checks)	some
This RFC	Kong plugin	gateway	central auth-service	none

User identity is brokered: in-house staff sign in via **Entra** (granted the `employee` role); external users self-register with email + password (`customer`). All users persist in Keycloak's Postgres.

Goal of this spike (questions to answer)

- Is the **community kong-oidc plugin** viable on our Kong version (OSS 3.x), or do we need Kong Enterprise's OIDC / a different gateway?
- What is the correct **issuer/hostname** strategy so tokens validate while the gateway reaches Keycloak internally and Entra's redirect-URI constraints hold?
- Does centralizing authorization in one service give acceptable **latency** and an acceptable **blast radius** (it becomes a hot path / SPOF)?
- What belongs in central policy vs. what must stay in the app (object-level authZ)?
- What is the operational cost (HA, sessions/Redis, secret management, plugin maintenance)?

Proposed architecture (RFC) — component view



```

| - inject header X-Userinfo: base64({ sub,email,name,roles }) |
| upstream (both apps) = auth-service [preserve_host: true] |
+=====+
|
| v
+===== auth-service :4000 (policy + proxy) =====+
| AUTHORIZATION (central POLICY table) + reverse proxy |
| - decode X-Userinfo -> per-host rule (App Two => role 'employee') |
| - allow -> set X-Auth-{Email,Name,Roles,Sub} -> proxy onward |
| - deny -> 403 page (app never reached) |
+=====+
| (root path) | (/api/*) |
| v | v |
| frontend1 / frontend2 (static) | backend1 / backend2 (echo only) |
|
Kong --(token / jwks / userinfo: keycloak:8080)--> Keycloak :8080
Kong --(sessions)--> Redis :6379 | Keycloak --> Entra ID (broker)
| Keycloak --> Postgres (users)

```

Decision boundary (the heart of the RFC): Kong answers *"are you logged in?"* (authentication). The auth-service answers *"may you reach this host/path?"* (authorization). Apps answer nothing — they trust headers set by the trusted edge.

Request lifecycle — sequence diagram



AuthN = the first round-trip through Keycloak + session creation. AuthZ = the single **POLICY** lookup on the authenticated request. A customer hitting App Two authenticates successfully, then receives **403** — proving the two concerns are separated.

The issuer / hostname problem (must-solve)

OIDC clients validate the token **iss**, so the URL the **browser** uses and the URL the **gateway** uses server-side must yield the same issuer — while Entra only allows plain-http redirect URLs for the literal host **localhost**. lua-resty-openidc has no per-request URL rewrite hook, so we feed it a **pre-built discovery table with split URLs**:

```
issuer / authorization_endpoint / end_session_endpoint --> http://localhost:8080 (browser-facing; iss matches)
token_endpoint / userinfo_endpoint / jwks_uri          --> http://keycloak:8080 (gateway-facing; reachable)
```

Open question for prod: behind real TLS + a public hostname this split collapses (one externally-resolvable issuer). Confirm the strategy for prod DNS so we don't ship a localhost-only hack. **SPIKE TASK**.

Code snippet — the central policy (the only "business logic" of authZ)

```
// auth-service: authZ is one declarative table + one check. Swap for OPA/Cedar later.
const POLICY = {
  'app-one.localhost': { appName: 'Customer Portal', requireRole: null }, // any authenticated user
  'app-two.localhost': { appName: 'Internal Tools', requireRole: 'employee' }, // employees only
};

app.use((req, res) => {
  const host = (req.headers.host || '').split(':')[0];
  const claims = JSON.parse(Buffer.from(req.headers['x-userinfo'], 'base64').toString()); // set by Kong
  const roles = claims.roles || [];
  const rule = POLICY[host];

  if (rule.requireRole && !roles.includes(rule.requireRole)) // <-- AUTHORIZATION
    return res.status(403).send(denyPage(rule, claims.email, roles));

  req.headers['x-auth-email'] = claims.email; // hand identity to the app
  req.headers['x-auth-roles'] = roles.join(',');
  proxy.web(req, res, { target: req.url.startsWith('/api') ? backend(host) : frontend(host) });
});
```

Key design decisions & trade-offs

Decision	Rationale	Trade-off / risk
OIDC at gateway (Kong plugin)	authN implemented once; apps in any language	community plugin needs a Kong-3.x port + version pinning
Central auth-service for authZ	one place for policy; uniform audit; zero-auth apps	extra hop; hot path; SPOF — must be HA
Sessions in Redis	lets the gateway scale horizontally	another stateful dependency to operate
Keycloak brokers to Entra	swap/add IdPs without app changes; one user store	Keycloak is itself critical infra
Coarse policy (host/role) at edge	fits gateway's visibility (line+headers)	object-level authZ ("owns this record?") stays in app

Spike tasks / investigation checklist

- ☐ Stand up the reference stack ([sso-demo-3](#)) and complete a real Entra login end-to-end (employee + customer).
- ☐ Confirm kong-oidc plugin viability on our pinned Kong OSS version; document the build (luarocks manifest workaround, 2.x→3.x port).
- ☐ Validate the issuer split locally; design the **production** issuer/DNS/TLS strategy.
- ☐ Measure added latency of the Kong→auth-service hop (p50/p95) under load.
- ☐ Define HA topology for auth-service + Redis + Keycloak; estimate cost.
- ☐ Decide policy engine: hardcoded table vs. **OPA/Rego** vs. **Cedar**; PoC the chosen one.
- ☐ Define the trust boundary enforcement (network policy / mTLS so apps can't be called directly).
- ☐ Map object-level authZ that must remain in apps; document the split contract (headers).
- ☐ Logout completeness (RP-initiated / id_token_hint) — current plugin version is limited.

Open questions & risks

#	Question / risk	Severity
1	Community kong-oidc maintenance burden; is Kong Enterprise OIDC or Envoy/oauth2-proxy a better long-term bet?	High
2	auth-service as SPOF/hot path — latency budget & HA story must be proven before adoption.	High
3	Production issuer/TLS strategy (the split is a local-dev device).	Med
4	Header trust boundary — forged <code>X-Userinfo</code> / <code>X-Auth-*</code> if a backend is reachable directly.	High
5	Policy externalization & testing (OPA/Cedar) — avoid policy drift / one-team coupling.	Med
6	Session store sizing & eviction; logout/SLO across the IdP.	Low

Acceptance criteria (Definition of Done for this spike)

- ☐ Written recommendation: **adopt** / **adopt-with-changes** / **reject**, with justification.
- ☐ All open questions above answered or converted into follow-up tickets.
- ☐ Latency numbers for the extra authZ hop, under representative load.
- ☐ Production issuer/TLS + HA topology documented (diagram).
- ☐ Decision on the policy engine, with a small PoC if not the hardcoded table.
- ☐ Trust-boundary enforcement approach defined (network policy / mTLS).
- ☐ Demo to the team; follow-up implementation epic drafted if "adopt".

Estimate & out of scope

Timebox: 3 days (5 pts).

Out of scope: production rollout, app migration, the policy-engine implementation itself, and SIEM/audit pipeline wiring — each becomes its own ticket if we proceed.

References

- Reference PoC: [sso-demo-3/](#) (README + runnable compose stack).
- Comparison of all three SSO patterns: [sso-demos-explained.pdf](#).
- kong-oidc (community), lua-resty-openidc, Keycloak identity brokering, Microsoft Entra ID app registration.
- Policy-engine options for follow-up: Open Policy Agent (Rego), AWS Cedar.